

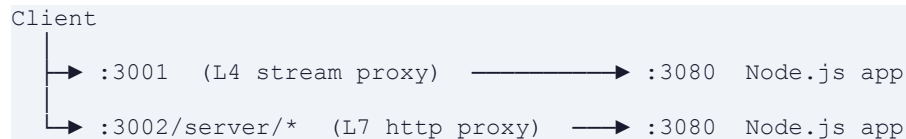
Nginx L4 vs L7 Proxy

Experiment on DigitalOcean Ubuntu Server

Overview

This experiment demonstrates the difference between Nginx operating at OSI Layer 4 (Transport) and Layer 7 (Application). A Node.js / Express service runs on port 3080 and echoes back every header and body it receives. Nginx forwards traffic to this service at two different ports using two different proxy strategies.

Architecture



L4 vs L7 — Feature Comparison

Feature	L4 — Transport Layer (stream)	L7 — Application Layer (http)
nginx config block	stream { }	http { }
Extra module needed?	Yes — libnginx-mod-stream	No — built in
Modify HTTP headers	✗ Not possible	✓ proxy_set_header
Headers passed to upstream	All original headers unchanged	All original + any added/modified headers
proxy_pass format	IP:PORT only (no scheme)	http://host:port or upstream name
URL path routing	✗ Not possible	✓ location /server/
Connection header (upstream)	keep-alive (transparent)	close (nginx re-opens connection)
Host header upstream sees	Original host:port as-is	Can be rewritten with proxy_set_header Host
Typical use-case	TCP/UDP passthrough, raw proxying	HTTP reverse proxy, load balancing, API gateway

Layer 4 Proxy (Transport Layer)

How it works

Nginx operates at the TCP level. It opens a TCP connection from the client, then opens a separate TCP connection to the upstream, and blindly pipes bytes between the two. It has no concept of HTTP — it cannot inspect, add, or remove headers.

nginx.conf — stream block

```
# Load the stream module first (apt install libnginx-mod-stream)
include /etc/nginx/modules-enabled/*.conf;

stream {
    server {
        listen 3001;

        # Must be IP:PORT — http:// scheme is NOT valid here
        proxy_pass 127.0.0.1:3080;
    }
}
```

Installing the stream module

The stream module is NOT bundled with the default nginx package on Ubuntu. Install it before use:

```
sudo apt install libnginx-mod-stream
sudo nginx -t && sudo systemctl reload nginx
```

Once installed, the package drops a .conf file into /etc/nginx/modules-enabled/ that loads the module. The include directive at the top of nginx.conf picks it up automatically.

Test request

```
curl -X POST http://167.172.87.210:3001/data \
-H 'test-header-1: test-header-1-value' \
-H 'Authorization: 1234' \
-H 'Content-Type: application/json' \
-d '{"hello": "world"}'
```

Observed headers at the upstream

The upstream sees the original client headers exactly as sent. Nginx does not add or modify anything:

```
"headers": {
  "test-header-1": "test-header-1-value",
  "authorization": "1234",
  "content-type": "application/json",
  "host": "167.172.87.210:3001",    // original host:port unchanged
  "connection": "keep-alive",      // TCP keep-alive maintained
  "content-length": "24"
}
```

Key observations

- No x-real-ip or x-forwarded-for — nginx cannot inject these at L4.
- The host header still shows port 3001 — the original value is untouched.
- Connection is keep-alive — the client TCP connection is transparently bridged.
- URL routing is impossible — /data is just bytes to nginx; all paths hit the same upstream.

Layer 7 Proxy (Application Layer)

How it works

Nginx operates at the HTTP level. It fully parses the incoming HTTP request, applies any header manipulation rules, then opens a new HTTP connection to the upstream. The client connection and the upstream connection are entirely separate — nginx is the middleman.

nginx.conf — http block

```
http {
    server {
        listen 3002;

        location /server/ {
            # Inject headers — only possible at L7
            proxy_set_header Host          $host;
            proxy_set_header X-Real-IP      $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header test           test-value;

            # http:// scheme is valid here.
            # Trailing slash strips the /server/ prefix (URI rewrite).
            proxy_pass http://localhost:3080/;
        }
    }
}
```

URL path routing & prefix stripping

The location `/server/` block matches any path starting with `/server/`. Because both the location and `proxy_pass` have a trailing slash, nginx rewrites the URI before forwarding:

```
Client sends:      POST /server/data
Upstream receives: POST /data      ← /server/ prefix stripped
```

This rewriting only works when both location and proxy_pass end with /. Removing the trailing slash from proxy_pass passes the full original path to the upstream.

Test request

```
curl -X POST http://167.172.87.210:3002/server/data \
-H 'test-header-1: test-header-1-value' \
-H 'Authorization: 1234' \
-H 'Content-Type: application/json' \
-d '{"hello": "world"}'
```

Observed headers at the upstream

The upstream sees the original client headers plus the injected ones:

```
"headers": {
  "host": "167.172.87.210",           // rewritten — port stripped
  "x-real-ip": "175.157.109.224",    // injected by nginx
  "x-forwarded-for": "175.157.109.224", // injected by nginx
  "test": "test-value",              // custom header injected
  "connection": "close",             // nginx closes upstream conn
}
```

```
"test-header-1": "test-header-1-value",
"authorization": "1234",
"content-type": "application/json"
}
```

Key observations

- x-real-ip and x-forwarded-for are present — nginx injected the real client IP.
- A custom test: test-value header was added — header manipulation works at L7.
- The host header no longer includes the port — nginx rewrote it to just the hostname.
- Connection is close — nginx terminates the client connection and opens a fresh one upstream.
- Path routing works — /server/data was rewritten to /data before forwarding.

Upstream Service

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

app.js

```
const express = require('express');
const app = express();
const PORT = 3080;

app.use(express.json());

app.post('/data', (req, res) => {
  res.status(200).json({
    status: 'success',
    message: 'Request received',
    timestamp: new Date().toISOString(),
    data: {
      body: req.body,
      params: req.params,
      headers: req.headers,
      query: req.query
    }
  });
});

app.listen(PORT, () => console.log(`Listening on ${PORT}`));
```

Quick Reference

```
# Install stream module (one-time, L4 only)
```

```
sudo apt install libnginx-mod-stream

# Validate config
sudo nginx -t

# Reload without downtime
sudo systemctl reload nginx

# Watch live logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```